AeroSim: Conception et implémentation d'une simulation interactive d'écoulement 3D par la méthode des panneaux

LOUIS LEJAILLE[1]

[1] *Projet personnel AeroSim*

Compiled November 4, 2026

AeroSim est une application de bureau développée en C#, .NET, WPF et HelixToolkit pour visualiser un écoulement d'air autour d'objets 3D. Le projet utilise une méthode des panneaux pour calculer un champ de vitesse, puis ce champ sert à déplacer des particules et à tracer des lignes de courant. Les modèles 3D sont décomposés en triangles. Sur chaque triangle, une source mathématique est définie, puis un solveur détermine son intensité de manière à limiter la pénétration du fluide dans l'obstacle. Pour rendre l'affichage plus rapide, les vitesses calculées sont ensuite enregistrées sur une grille 3D. Cette revue présente les choix techniques, les étapes de développement, les équations utilisées et les premiers diagnostics. Elle décrit également les limites observées sur un modèle d'avion plus complexe. AeroSim reste un projet personnel d'expérimentation et de visualisation, et non un outil de calcul aérodynamique industriel validé.

1. INTRODUCTION ET ENVIRONNEMENT TECHNOLOGIQUE

Simuler visuellement un écoulement autour d'un objet 3D demande de réunir plusieurs éléments : un modèle géométrique, une méthode de calcul des vitesses, un système capable de déplacer des particules et un affichage 3D. L'objectif d'AeroSim est de faire fonctionner ces éléments dans une même application. Le projet consiste donc surtout à comprendre une méthode numérique existante et à l'intégrer dans un logiciel interactif. Les trajectoires affichées aident à observer le comportement du champ, mais elles ne suffisent pas à prouver que la simulation est physiquement exacte.

A. Présentation du projet AeroSim

AeroSim permet actuellement de tester trois types d'obstacles : un cube, une sphère et un avion *low-poly* importé à partir d'un fichier OBJ. L'utilisateur règle la vitesse du vent en km/h. Celle-ci est convertie en m/s pour les calculs. Dans la version présentée ici, le vent est dirigé suivant l'axe Y et l'angle d'incidence est fixé à $0°$. L'application affiche le modèle 3D, des particules en mouvement, leurs traces et des lignes de courant. Une fenêtre supplémentaire présente les informations sur le maillage, les temps de calcul, les erreurs numériques mesurées et les statistiques des particules. L'idée est de pouvoir suivre toutes les étapes, du chargement de l'objet jusqu'à l'affichage de l'écoulement.



Fig. 1. Interface d'AeroSim montrant l'écoulement autour d'un modèle 3D d'avion.

B. Choix technologiques : C#, .NET, WPF et HelixToolkit

C# et .NET ont été retenu pour développer le moteur de simulation. WPF permet de construire l'interface de bureau, de gérer les réglages et de mettre à jour l'affichage. Pour la partie 3D, HelixToolkit.Wpf fournit les outils de visualisation et permet notamment de charger des fichiers OBJ avec `ModelImporter`. Les classes `BoxVisual3D` et `SphereVisual3D` servent aussi à créer les obstacles simples. Ces outils permettent de réunir le calcul et la visualisation dans une seule application. En contrepartie, il faut faire attention aux performances, surtout lorsque les modèles comportent plusieurs milliers de panneaux.

2. PROBLÉMATIQUE ET CHOIX DE LA MÉTHODE NUMÉRIQUE

Le principal problème est de calculer un champ de vitesse qui change en fonction de la présence d'un obstacle. Avec un vent uniforme, les particules avanceraient toutes dans la même direction et pourraient traverser l'objet. Il faut donc une méthode qui prenne en compte la surface du modèle 3D et modifie les vitesses

autour de celui-ci. Pour AeroSim, cette méthode devait rester suffisamment simple à étudier et à intégrer dans une application interactive.

A. Simulation d'un écoulement autour d'un obstacle 3D

Dans le modèle utilisé, l'air ne doit pas passer à travers la surface solide. Cela signifie que la vitesse perpendiculaire à la paroi doit être nulle. En revanche, une vitesse parallèle à la surface reste possible. Cette différence vient du fait que le modèle ne prend pas en compte la viscosité ni la condition de non-glissement. Une fois le champ calculé, les particules peuvent être déplacées à partir de la vitesse obtenue à leur position. Le calcul du champ et le mouvement des particules sont donc deux étapes séparées, ce qui permet de les tester indépendamment.

B. Panorama des approches numériques

Il existe plusieurs façons de simuler un écoulement. Les méthodes de CFD utilisant un maillage du volume, comme les volumes finis, permettent notamment d'étudier des effets liés à la viscosité. Elles demandent cependant un maillage de l'espace autour de l'objet et des calculs plus complexes. La méthode *lattice Boltzmann* est une autre approche possible. La méthode des panneaux suit une logique différente : dans le cadre d'un écoulement potentiel, le calcul repose principalement sur la surface de l'obstacle.

C. Justification de la méthode des panneaux et limites

La méthode des panneaux a été choisie car elle permet de travailler directement à partir des triangles d'un modèle 3D, tout en donnant accès à un calcul de vitesse dans l'espace. Elle est également assez bien documentée, notamment dans des publications techniques de la NASA [1, 2]. Ce choix ne signifie pas qu'elle est plus performante que les méthodes récentes. Dans AeroSim, chaque triangle porte uniquement une source d'intensité constante. Cette version ne modélise pas la viscosité, la couche limite, le décollement ou un sillage visqueux réaliste. Elle n'utilise pas non plus de doublets ou de circulation. Elle ne peut donc pas servir à calculer correctement la portance réelle d'une aile. L'objectif est avant tout de comprendre et de mettre en œuvre une première méthode de calcul d'écoulement.

3. FONDEMENTS THÉORIQUES DE LA MÉTHODE DES PANNEAUX

La méthode utilisée dans AeroSim repose sur un écoulement potentiel incompressible. Son fonctionnement peut être résumé en quatre étapes : découper l'objet en panneaux, calculer l'influence des panneaux les uns sur les autres, déterminer les intensités des sources et utiliser ces résultats pour obtenir une vitesse en un point de l'espace.

A. Hypothèses et formulation du problème

On suppose que l'écoulement est irrotationnel dans le domaine fluide. La vitesse peut alors être représentée à partir d'un potentiel ϕ, avec $\mathbf{V} = \nabla\phi$. En supposant aussi le fluide incompressible, ce potentiel respecte l'équation de Laplace $\nabla^2\phi = 0$. Dans AeroSim, la vitesse totale correspond à la vitesse du vent incident $\mathbf{V}_\infty$, à laquelle s'ajoutent les vitesses créées mathématiquement par les panneaux. La condition que l'on cherche à respecter sur l'obstacle est :

$$\mathbf{V}(\mathbf{x}_i) \cdot \mathbf{n}_i = 0, \tag{1}$$

Ici, $\mathbf{x}_i$ représente le centre du panneau i et $\mathbf{n}_i$ sa normale, c'est-à-dire la direction perpendiculaire à sa surface. Le produit scalaire exprime la partie de la vitesse qui pointe dans cette direction. Le résultat doit être nul pour empêcher le fluide de traverser la paroi. Dans le programme, cette condition est imposée aux centres des panneaux, et non à chaque point de leur surface (figure 2).

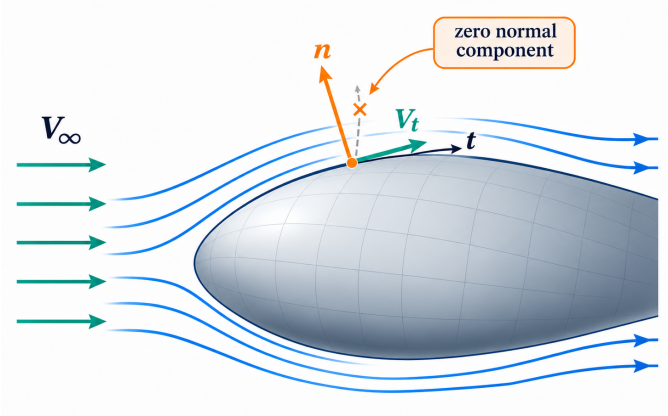


Fig. 2. Condition de non-pénétration : la vitesse au contact de la surface doit être tangentielle à la paroi.

B. Discrétisation de la surface en panneaux

Le premier travail consiste à récupérer les triangles du modèle 3D. `MeshExtractor.cs` parcourt les objets de la scène, lit les indices de `MeshGeometry3D` et applique leurs transformations pour obtenir les coordonnées dans la scène. Chaque triangle est ensuite représenté par un panneau dans `SurfaceMesh.cs` et `SurfacePanel.cs`. À partir des trois sommets A, B et C, le programme calcule :

$$P = (A + B + C)/3, \tag{2}$$

$$S = \tfrac{1}{2}\|(B - A) \times (C - A)\|, \tag{3}$$

$$\mathbf{n} = \frac{(B - A) \times (C - A)}{\|(B - A) \times (C - A)\|}. \tag{4}$$

P est le centre du triangle, utilisé comme point de calcul. S correspond à son aire, et $\mathbf{n}$ est sa normale unitaire. Le produit vectoriel sert à trouver une direction perpendiculaire au triangle ; sa norme permet également d'en déduire l'aire. Le programme ignore les coordonnées invalides et les triangles dégénérés. L'ordre des sommets est important car il détermine le sens de la normale. Pour un cube de côté 10, on obtient par exemple 12 panneaux triangulaires et une surface totale de 600 unités carrées. La figure 3 présente ces grandeurs.

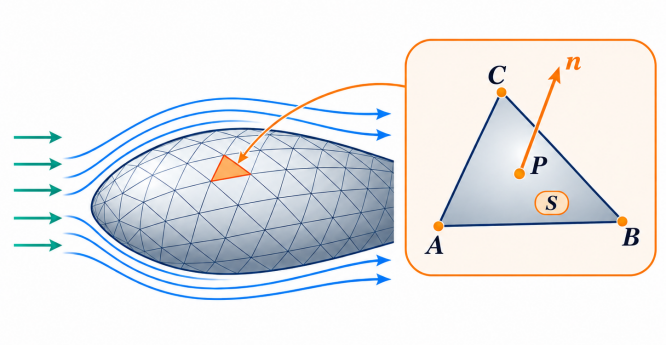


Fig. 3. Découpage d'une surface en panneaux triangulaires et principales grandeurs géométriques.

C. Conditions aux limites et coefficients d'influence

Chaque panneau j possède une intensité de source inconnue σ_j. Cette source est un élément mathématique du modèle : elle sert à modifier le champ de vitesse autour de l'objet. Pour savoir comment les panneaux agissent entre eux, on calcule un coefficient A_{ij}. Il décrit l'effet d'une source unitaire placée sur le panneau j sur la vitesse normale au centre du panneau i. Dans `PanelInfluence.cs`, cet effet est approché en évaluant trois points du triangle : $(4A + B + C)/6$, $(A + 4B + C)/6$ et $(A + B + 4C)/6$. Les contributions sont pondérées par $S/(12\pi)$ et utilisent le noyau suivant :

$$\frac{\mathbf{r}}{(\|\mathbf{r}\|^2 + \varepsilon^2)^{3/2}}, \qquad \varepsilon = \max(10^{-6}, 0.01\sqrt{S}). \tag{5}$$

Le vecteur $\mathbf{r}$ relie le point d'évaluation au point où l'on calcule l'influence. Le paramètre ε évite que le calcul devienne trop grand lorsque la distance est très faible. Il rend le calcul plus stable, mais ajoute une approximation. Pour l'influence d'un panneau sur lui-même, le coefficient diagonal est fixé à $A_{ii} = 0.5$. Cette valeur correspond au terme de saut de la vitesse normale d'une couche de sources, avec une convention de signe qui dépend de l'orientation des normales et du côté de la surface considéré.

D. Construction et résolution du système linéaire

Après avoir calculé les influences, il reste à trouver les intensités σ_j de tous les panneaux. Pour N panneaux, on obtient N inconnues et N équations. Le système est écrit sous la forme :

$$\underbrace{A\sigma}_{\text{sources induites}} = \underbrace{\mathbf{b}}_{\text{vent incident}}, \qquad b_i = -\mathbf{V}_\infty \cdot \mathbf{n}_i. \tag{6}$$

La matrice A contient les influences entre les panneaux, le vecteur σ contient les intensités recherchées et $\mathbf{b}$ représente la part du vent incident qu'il faut compenser pour respecter la condition de non-pénétration. Les coefficients hors diagonale sont calculés avec $A_{ij} = \mathbf{v}_{ij} \cdot \mathbf{n}_i$, où $\mathbf{v}_{ij}$ est la vitesse produite par une source unitaire sur le panneau j au centre du panneau i. `PanelMethodSolver.cs` résout ce système par élimination de Gauss avec pivot partiel, en double précision. Si un pivot est inférieur à 10^{-12}, le calcul est interrompu par une exception. Le programme calcule également le résidu $\mathbf{r} = A\sigma - \mathbf{b}$, qui indique à quel point les valeurs trouvées respectent les équations assemblées. Un petit résidu signifie que le système est bien résolu, mais pas forcément que le champ est physiquement exact. Enfin, le stockage de la matrice demande $O(N^2)$ valeurs et l'élimination de Gauss dense a un coût en $O(N^3)$. Cela explique pourquoi les modèles comportant beaucoup de panneaux deviennent coûteux à traiter (figure 4).

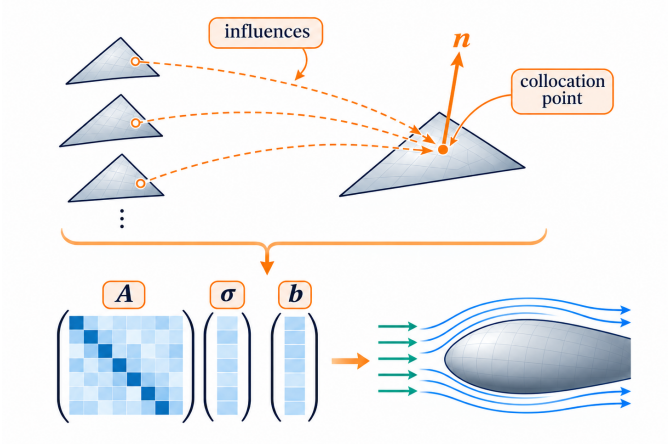


Fig. 4. Influence des panneaux sur un point de collocation et construction du système linéaire.

E. Calcul du champ de vitesse

Une fois les intensités σ_j connues, AeroSim peut calculer la vitesse de l'air à une position $\mathbf{x}$ avec `PanelFlowField.cs` :

$$\mathbf{V}(\mathbf{x}) = \mathbf{V}_\infty + \sum_{j=1}^{N} \sigma_j \, \mathbf{v}_j(\mathbf{x}). \tag{7}$$

L'équation additionne le vent incident et les contributions des différents panneaux, chacune étant multipliée par l'intensité de sa source. Un calcul direct demande donc de parcourir les N panneaux pour chaque position interrogée, soit un coût en $O(N)$. Grâce à l'interface `IFlowField`, le reste de la simulation peut utiliser un champ uniforme, un champ direct ou un champ mis en cache sans changer le code du système de particules. Tant que la géométrie et le vent restent identiques, ce champ ne change pas au cours du temps.

4. ARCHITECTURE ET IMPLÉMENTATION D'AEROSIM

Le projet est organisé de manière à séparer les tâches principales : gérer les obstacles, construire les panneaux, calculer le champ de vitesse, déplacer les particules et afficher la scène. Cette séparation aide aussi à rechercher les erreurs. Par exemple, une particule qui traverse un obstacle peut révéler un problème dans le champ, dans son interpolation ou dans le déplacement de la particule, sans que le solveur soit nécessairement responsable.

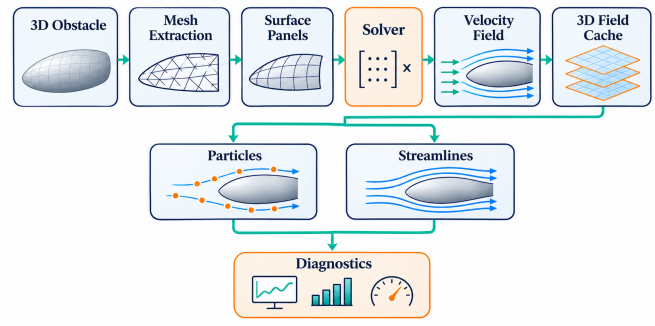


Fig. 5. Principales étapes de calcul et de visualisation dans AeroSim.

A. Organisation et architecture logicielle

`ObstacleManager.cs` gère le choix de l'objet affiché. `MeshExtractor.cs`, `SurfaceMesh.cs` et `SurfacePanel.cs` récupèrent la géométrie et construisent les panneaux. `PanelInfluence.cs` et `PanelMethodSolver.cs` calculent les influences et résolvent les intensités des sources. `PanelFlowField.cs` fournit ensuite les vitesses. D'autres classes gèrent les particules et leur évolution, tandis que `StreamlineGenerator.cs` calcule les lignes de courant. Enfin, `SimulationDataWindow.xaml.cs` présente les diagnostics et les temps de calcul. On distingue donc une phase de préparation du champ, relancée lorsque les paramètres changent, et une phase d'affichage mise à jour pendant l'animation.

B. Simulation des particules et recyclage

La simulation utilise 10 000 particules. Au départ, elles sont réparties de façon assez régulière dans la direction du vent, puis légèrement décalées de manière aléatoire. Ce décalage, appelé *jitter*, évite des alignements trop artificiels. Une graine aléatoire fixe, 12345, permet de retrouver la même suite de tirages dans les mêmes conditions. Pour déplacer les particules, AeroSim utilise la méthode d'Euler explicite avec un pas de temps fixe de 0.02 s. L'accumulateur limite à 0.1 s le temps pris en compte pour une image afin de ne pas accumuler trop de retard. Certains mouvements sont divisés en sous-pas pour limiter la distance parcourue d'un seul coup. Cette distance est basée sur 0.55 fois la plus petite cellule du cache, avec au maximum huit sous-pas.

Une particule est replacée dans le domaine lorsqu'elle en sort, lorsqu'elle atteint sa durée de vie ou lorsqu'elle reste bloquée ou est détectée dans l'obstacle. La durée de vie nominale est de 55 s, avec une variation de $\pm 35\,\%$, et le blocage est recherché sur une période de 3 s. Le logiciel compte séparément les différentes causes de recyclage. Ce mécanisme sert surtout à conserver une visualisation exploitable : il ne constitue pas une gestion complète des collisions et n'empêche pas toutes les traversées de surface. Chaque particule conserve aussi ses dix dernières positions pour afficher sa trace.

C. Chargement des modèles 3D et extraction des panneaux

Pour les premiers tests, le cube est créé avec `BoxVisual3D`, avec des dimensions de $10 \times 10 \times 10$, et la sphère avec `SphereVisual3D`, de rayon 5, avec `ThetaDiv=8` et `PhiDiv=6`. L'avion *low-poly* est importé depuis un fichier OBJ à l'aide de `ModelImporter`, puis positionné dans la scène. Le programme parcourt ensuite les objets 3D pour récupérer leurs triangles dans le bon système de coordonnées. Il écarte les triangles invalides et calcule les données nécessaires à la création des panneaux. L'objet affiché et le maillage utilisé pour les calculs proviennent donc de la même géométrie. Cependant, un modèle qui s'affiche correctement n'est pas forcément fermé ou correctement orienté pour la méthode des panneaux.

D. Intégration du solveur et calcul de l'écoulement

Lorsqu'un changement impose de recalculer l'écoulement, AeroSim récupère les triangles, construit les panneaux, calcule leurs influences, résout les intensités des sources et prépare le champ de vitesse. Pour éviter des temps de calcul trop importants, la simulation est désactivée au-delà de 5 000 panneaux, même si l'objet peut encore être affiché. La vitesse saisie dans l'interface est divisée par 3.6 pour passer de km/h à m/s. Dans l'état du projet présenté ici, le solveur utilise un vent suivant $+Y$ et un angle d'incidence nul. Le calcul des intensités de sources et l'utilisation du champ pour animer les particules restent deux opérations distinctes.

E. Visualisation des particules et des lignes de courant

Les particules représentent des points qui avancent au cours du temps en suivant la vitesse locale. Les lignes de courant donnent une autre vue du champ : elles sont calculées à partir de 77 positions de départ placées sur une grille de 11×7 points en amont de l'obstacle. `StreamlineGenerator.cs` utilise la méthode de Runge–Kutta d'ordre quatre (RK4), appliquée à la direction de vitesse normalisée, avec au plus 400 pas. Les particules et les lignes de courant utilisent donc des méthodes d'intégration différentes. Elles peuvent représenter le même champ stationnaire sans produire exactement les mêmes tracés numériques.

Pour mieux voir les accélérations et les ralentissements, la visualisation utilise cinq classes de couleur selon le rapport entre la vitesse locale et celle du vent incident : moins de 0.7, de 0.7 à 0.95, de 0.95 à 1.05, de 1.05 à 1.25 et au moins 1.25. La couleur d'un segment de ligne de courant dépend de la vitesse calculée au milieu de ce segment. Les éléments de même classe sont regroupés pour limiter le nombre d'objets à afficher. La mise à jour visuelle des particules est plafonnée à 120 fois par seconde, et les différents modes d'affichage peuvent être activés séparément.

F. Optimisations et mise en cache du champ de vitesse

Un calcul direct de vitesse demande de prendre en compte tous les panneaux. Le refaire pour chaque particule serait donc trop coûteux, surtout sur les modèles complexes. Pour éviter cela, AeroSim calcule à l'avance les vitesses sur une grille 3D de taille $n_x \times n_y \times n_z$. Lorsqu'une particule se trouve entre les points de cette grille, sa vitesse est estimée par interpolation trilinéaire à partir des huit nœuds voisins. Une fois le cache construit, cette interpolation prend un temps presque constant, indépendamment du nombre de panneaux. En dehors de la grille, le programme utilise le vent incident. Les dimensions du cache sont calculées à partir de la boîte englobante de l'objet et de sa plus grande dimension L. Le pas de référence est $L/16$, avec un maximum de 150 000 nœuds. Une grille moins fine coûte moins cher à construire, mais risque de perdre des variations de vitesse près de la surface (figure 6).

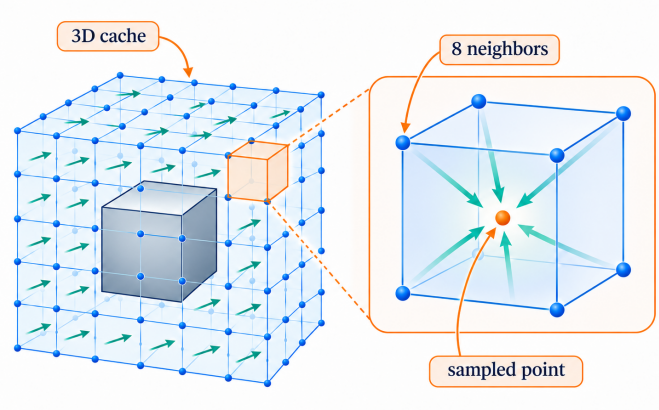


Fig. 6. Cache 3D du champ de vitesse et interpolation à partir des huit nœuds voisins.

Le programme doit aussi déterminer si une particule se trouve à l'intérieur d'un obstacle. Il commence par vérifier certaines propriétés du maillage, comme sa fermeture et l'orientation de ses triangles. Il utilise ensuite des tests de rayons suivant trois axes et un vote majoritaire pour estimer si le point est à l'intérieur. Des index spatiaux réduisent le nombre de triangles à tester. Si le maillage n'est pas considéré comme fiable, le logiciel utilise une approximation basée sur la boîte englobante. Cette solution est plus simple, mais peut aussi classer comme intérieurs des points qui se trouvent en réalité à l'extérieur de l'objet.

5. DÉMONSTRATION, VALIDATION ET CONCLUSION

Pour évaluer AeroSim, j'utilise plusieurs types de contrôles : la vérification des données géométriques, les résidus du solveur, la comparaison entre le champ direct et le cache, ainsi que le suivi des particules. Ces indicateurs permettent de repérer des problèmes et de mieux comprendre le comportement du logiciel. Ils ne remplacent toutefois pas une validation complète par comparaison à une solution analytique ou à un logiciel de référence.

A. Démonstration de l'application

Les essais réalisés utilisent trois obstacles de complexité différente. Le cube constitue le cas le plus simple : ses 12 triangles et sa surface totale de 600 unités carrées sont faciles à vérifier. La sphère, représentée par 80 triangles, permet d'observer le contournement d'une forme courbe. Enfin, l'avion *low-poly*, composé d'environ 3 600 panneaux lors des essais, permet de tester le logiciel sur une géométrie beaucoup plus complexe. Aucun comportement anormal n'a été observé visuellement sur le cube et la sphère dans les essais réalisés. Ce constat reste cependant différent d'une validation physique chiffrée.

B. Cas de test et validation des résultats

Un premier contrôle concerne la résolution du système linéaire. Le programme calcule le résidu $A\sigma - \mathbf{b}$: il mesure la différence entre les deux côtés des équations après résolution. Un autre test vérifie la vitesse normale sur des panneaux échantillonnés, en tenant compte du terme de saut 0.5σ à la surface. Le résidu RMS donne une valeur représentative de l'ensemble des erreurs, tandis que le résidu maximal indique le plus grand écart observé. Lorsque la fenêtre affiche 0.0000 m/s, cela peut simplement vouloir dire que la valeur est trop petite pour apparaître avec quatre décimales.

Un deuxième contrôle compare les vitesses calculées directement avec celles obtenues par interpolation du cache à des positions fixes. Pour l'essai complexe documenté, 15 sondes donnent une erreur RMS de 0.0037 m/s et une erreur maximale de 0.0098 m/s, avec un vent incident de 5.56 m/s. Ces valeurs indiquent une faible différence aux endroits testés, mais ne prouvent pas que l'erreur est faible partout, notamment près des panneaux. Le programme mesure aussi si le champ direct reste constant dans le temps et recherche les vitesses extrêmes sur une grille de points.

Enfin, le suivi des particules permet de mesurer les vitesses rencontrées, leur composante verticale moyenne, le nombre de sous-pas ainsi que les différentes causes de recyclage. Le nombre de particules considérées comme présentes dans l'obstacle est également affiché. Sur l'essai de l'avion *low-poly*, certaines particules ont été détectées dans le modèle et des vitesses particulièrement élevées ont été observées. Plusieurs causes sont possibles : qualité du maillage, calcul direct du champ, interpolation du cache ou méthode de déplacement des particules. Les données actuelles ne permettent pas encore de choisir une seule explication. Les captures et leurs intitulés devront également être vérifiés avant de présenter ces mesures comme des résultats définitifs.

C. Limites et perspectives

La première limite vient du modèle physique choisi. Avec uniquement des sources constantes, sans circulation ni condition de Kutta, AeroSim ne peut pas représenter correctement la portance d'une aile. Le modèle ne calcule pas non plus la viscosité, la couche limite ou le décollement de l'écoulement. D'autres limites viennent de l'implémentation : le calcul des influences utilise trois points par triangle et un noyau régularisé ; le solveur repose sur une résolution dense en $O(N^3)$ sans parallélisation. Le cache peut lisser les variations importantes du champ près des parois. Enfin, la méthode d'Euler peut faire traverser une surface à une particule entre deux positions calculées. Le recyclage corrige alors l'affichage, mais pas le problème physique lui-même.

Pour aller plus loin, il faudra d'abord vérifier la qualité des maillages, en particulier leur fermeture et l'orientation des normales. Il serait aussi utile de comparer les champs direct et interpolé près des surfaces, puis d'étudier l'effet du nombre de panneaux et de la résolution du cache. Une étape importante serait de comparer les résultats d'AeroSim à une solution connue, par exemple l'écoulement potentiel autour d'une sphère. Des améliorations des performances, par parallélisation ou avec des méthodes de résolution plus adaptées aux grands systèmes, pourront être envisagées après ces tests.

D. Conclusion

AeroSim m'a permis de réunir plusieurs domaines dans un même projet : développement C#, manipulation de géométries 3D, calcul numérique et visualisation interactive. La méthode des panneaux fournit un champ de vitesse à partir de la surface de l'obstacle ; ce champ est ensuite mis en cache et utilisé pour déplacer les particules et tracer les lignes de courant. Les essais sur les formes simples constituent une première vérification du fonctionnement. Le modèle d'avion montre en revanche que la gestion des géométries complexes et la fiabilité du champ demandent encore du travail. La prochaine étape est donc de mieux mesurer les erreurs et de comparer les résultats à des cas

de référence, afin de distinguer les limites du modèle de celles de son implémentation.

REFERENCES

1. F. T. Johnson, “A General Panel Method for the Analysis and Design of Arbitrary Configurations in Incompressible Flows,” NASA Contractor Report CR-3079 (1980). https://ntrs.nasa.gov/citations/19800015776
2. L. L. Erickson, “Panel Methods: An Introduction,” NASA Technical Paper 2995 (1990). https://ntrs.nasa.gov/citations/19910009745
3. “Lift to Drag Ratio” NASA Glenn Research Center https://www1.grc.nasa.gov/beginners-guide-to-aeronautics/lift-to-drag-ratio/